

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Реверс исполняемого файла»

Выполнили:

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Реверс исполняемого файла.....	4
1.1 Первичное исследование	4
1.2 Анализ исполняемого файла (реверс).....	4
1.2.1 Динамический анализ и формирование гипотезы (x32dbg)	4
1.2.2 Статический анализ и поиск функции проверки (IDA Pro)	5
1.2.3 Локализация функции-обработчика	5
1.2.4 Анализ алгоритма шифрования	6
1.3 Восстановление пароля.....	9
1.4 Проверка результата.....	10
1.5 Выполнение дополнительных заданий.....	11
1.5.1 Вход с любым паролем (x32dbg).....	11
1.5.2 Снятие блокировки после неверных попыток ввода	12
1.5.3 Разработка алгоритма подбора пароля (Брутфорс).....	13
Заключение.....	16
Список использованных источников.....	17

ВВЕДЕНИЕ

Цель работы – исследовать предоставленный исполняемый файл LabReverse.exe, провести его анализ для восстановления логики работы алгоритма проверки пароля и последующего вычисления корректного пароля для успешной аутентификации.

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести первичное исследование функциональности программы.
- С использованием специализированных инструментов найти в коде программы ключевые строки и константы.
- Определить функции и строки данных, используемые в программе.
- Локализовать функцию, ответственную за генерацию пароля.
- Декомпилировать и проанализировать ассемблерный код для восстановления алгоритма шифрования.
- Разработать скрипт для выполнения обратного преобразования и вычисления исходного пароля.

1 РЕВЕРС ИСПОЛНЯЕМОГО ФАЙЛА

1.1 Первичное исследование

Так как контроль учетных записей (UAC) в ОС Windows блокирует исполнение файла `LabReverse.exe`, все дальнейшие операции с файлом выполнялись в изолированной среде «Песочница Windows».

Первичное исследование поведения программы показало следующее:

- При запуске отображается строка-подсказка: Секрет: `px4wp466`.
- Программа требует ввода пароля длиной ровно 8 символов. При любой другой длине выводится сообщение об ошибке.
- После двух неудачных попыток ввода пароля правильной длины программа временно блокирует возможность ввода на 10 секунд.

Эти наблюдения позволили сделать вывод, что "секрет" является ключом или частью алгоритма, а искомый пароль имеет фиксированную 8-символьную длину.

1.2 Анализ исполняемого файла (реверс)

Анализ проводился в два этапа: первичный динамический анализ для формирования гипотез и последующий углубленный статический анализ для их подтверждения и восстановления алгоритма.

1.2.1 Динамический анализ и формирование гипотезы (x32dbg)

Первоначальное исследование проводилось с использованием отладчика `x32dbg`. Путем установки аппаратных точек останова на строку-секрет `px4wp466` было замечено, что обращения к этим данным происходят на ранних этапах запуска программы – после ее инициализации, но еще до отображения главного окна. Это позволило выдвинуть гипотезу: программа предварительно вычисляет правильный пароль или его зашифрованный эталон при старте, а не в момент нажатия кнопки.

Дальнейший анализ памяти в отладчике позволил обнаружить рядом со строкой-секретом еще одну ключевую строку данных – `INAQI=9D`. Учитывая ее длину (8 символов) и случайный набор символов, было сделано предположение, что это и есть заранее вычисленный зашифрованный эталон пароля.

1.2.2 Статический анализ и поиск функции проверки (IDA Pro)

Для подтверждения гипотезы и детального изучения алгоритма проводился статический анализ с помощью дизассемблера IDA Pro.

С помощью просмотра всех строковых литералов в файле (View -> Open subviews -> Strings) были повторно идентифицированы ранее обнаруженные строки, что подтвердило их статическое размещение в секции .data программы.

.text:0065B1E8	0000001E	C	System::Sysutils::EExternal *
.data:0065D000	00000047	C	Embarcadero RAD Studio - Copyright 2015 Embarcadero Technologies, Inc.
.data:0065D29D	00000009	C	IHAqI=9D
.data:0065D2DD	00000009	C	00000000
.data:0065D2E6	00000009	C	00000000
.data:0065D2F0	00000009	C	00000000
.data:0065D2F9	00000009	C	00000000
.data:0065D314	0000000F	C	Micro-Software
.data:0065D340	00000009	C	px4wp466
.data:0065D349	0000000F	C	Micro-Software
.data:0065DC9C	0000000A	C	\bFormShow
.data:0065DF3B	0000000F	C	Micro-Software
.data:0065EC5C	0000001E	C	Runtime error at 00000000
.data:0065F83A	00000009	C	us-ascii
.data:0065F851	00000010	C	csunicode11utf7

Рисунок 1 – Ключевые строки px4wp466 и IHAqI=9D в памяти программы

1.2.3 Локализация функции-обработчика

Для определения кода, отвечающего непосредственно за проверку пароля, был выполнен анализ перекрестных ссылок. В секции .data была найдена таблица опубликованных методов VCL (Visual Component Library), которая связывает события пользовательского интерфейса с функциями-обработчиками.

.data:0065DC71	db 0	
.data:0065DC72	dd offset @Iwmain@TFormIWMain@actnShutdownExecute\$qqrp14System@TObject	
.data:0065DC76 aButton2click	db 12, 'Button2Click'	
.data:0065DC83	db 13h	
.data:0065DC84	db 0	
.data:0065DC85	dd offset sub_40330C	
.data:0065DC89 aButton1click	db 12, 'Button1Click'	
.data:0065DC96	db 0Fh	
.data:0065DC97	db 0	
.data:0065DC98	dd offset sub_404400	
.data:0065DC9C aFormshow	db 8, 'FormShow', 0	
.data:0065DCA6	db 0	; Unknown
.data:0065DCA7 asc_65DCA7	db 6, 'TForm1'	; DATA XREF: .data:0065DA9C↑o
.data:0065DCAE unk_65DCAE	db 3	; DATA XREF: .data:0065DC1A↑o
.data:0065DCAF	db 0	
.data:0065DCB0	dd offset off_5413D4	

Рисунок 2 – Фрагмент таблицы VCL, связывающий событие

Button1Click с функцией sub_40330C

Как видно из листинга (Рисунок 2 –), событие `Button1Click` (нажатие на кнопку проверки) напрямую связано с адресом функции `sub_40330C`. Это однозначно идентифицирует ее как основную функцию, отвечающую за логику проверки пароля при взаимодействии с пользователем.

1.2.4 Анализ алгоритма шифрования

С помощью декомпилятора Hex-Rays (клавиша F5 в IDA Pro) был получен C-подобный псевдокод функции `sub_40330C`. Детальный анализ этого кода позволил полностью восстановить алгоритм.

Алгоритм выполняет следующие действия:

1) Проверка длины пароля:

В самом начале функция получает введенный пользователем текст и проверяет его длину. Если длина не равна 8, выводится сообщение «Неверная длинна пароля».

```

105 char v104[4]; // [esp+114h] [ebp-4h] BYREF
106
107 __InitExceptBlockLDT();
108 sub_65B454(v104, &unk_65D2C0);
109 sub_65B524(&unk_65D220, v104);
110 System::UnicodeString::~UnicodeString((System::UnicodeString *)v104);
111 unknown_libname_487(v103);
112 Vcl::Controls::TControl::GetText(*(Vcl::Controls::TControl **)(a1 + 964)); // Получаем текст из элемента управления (поля ввода)
113 v40 = sub_4042D4(v103) != 8; // Вызываем аналог strlen и сравниваем с 8
114 System::UnicodeString::~UnicodeString((System::UnicodeString *)v103);
115 if ( v40 )
116 {
117     v1 = (_DWORD *)sub_65B454(v102, &unk_65D2C1); // Вывод сообщения "Неверная длинна пароля"
118     unknown_libname_5177(*v1);
119     System::UnicodeString::~UnicodeString((System::UnicodeString *)v102);
120 }
121 else

```

Рисунок 3 – Проверка длины введенной строки

2) Цикл шифрования:

Если длина корректна, программа входит в основной цикл `for ( j = 1; j < 9; ++j )`, который выполняется 8 раз (Рисунок 4 –). Внутри цикла для каждого `j`-го символа введенного пароля и `j`-го символа секрета (`px4wr466`) происходят следующие преобразования:

а) Преобразование символов в числовые значения.

Программа берет ASCII-код символа пароля и символа секрета, после чего из каждого кода вычитается константа 48 (ASCII-код символа 0).

```

123     for ( i = 0; i < 8; ++i )
124     {
125         dword_65D224 += i;
126         if ( dword_65D224 > 7 )
127             dword_65D224 -= 8;
128     }
129     sub_4042F4();
130     sub_65B454(v101, &unk_65D2D8);
131     sub_65B454(v100, &unk_65D2D9);
132     sub_65B454(v99, &unk_65D2DA);
133     sub_65B454(v98, &unk_65D2DB);
134     v58 = v57 + 4;
135     for ( j = 1; j < 9; ++j )
136     {
137         sub_65B454(v96, &unk_65D2DC);
138         sub_65B524(v99, v96);
139         System::UnicodeString::~UnicodeString((System::UnicodeString *)v96);
140         sub_65B454(v95, a00000000);
141         sub_65B524(v98, v95);
142         System::UnicodeString::~UnicodeString((System::UnicodeString *)v95);
143         unknown_libname_487(v94);
144         Vcl::Controls::TControl::GetText(*(Vcl::Controls::TControl **)(a1 + 964));
145         v53 = *(unsigned __int16 *)sub_404308((System::UnicodeString *)v94, j);
146         System::UnicodeString::~UnicodeString((System::UnicodeString *)v94);
147         v54 = v53 - 48;
148         if ( v54 )
149         {
150             for ( ; v54 != 1; v54 /= 2 )
151             {
152                 sub_404330(v92, v54 % 2);

```

Рисунок 4 – Фрагмент кода для символа пароля

```

187     System::UnicodeString::~UnicodeString((System::UnicodeString *)v87);
188     unknown_libname_487(v86);
189     Vcl::Controls::TControl::GetText(*(Vcl::Controls::TControl **)(a1 + 976));
190     v48 = *(unsigned __int16 *)sub_404308((System::UnicodeString *)v86, j);
191     System::UnicodeString::~UnicodeString((System::UnicodeString *)v86);
192     v49 = v48 - 48;
193     if ( v49 )
194     {
195         for ( ; v49 != 1; v49 /= 2 )
196         {
197             sub_404330(v84, v49 % 2);
198             v5 = unknown_libname_487(v83);
199             System::UnicodeString::operator+(v5, v101, v84);
200             sub_65B524(v101, v83);
201             System::UnicodeString::~UnicodeString((System::UnicodeString *)v83);
202             System::UnicodeString::~UnicodeString((System::UnicodeString *)v84);
203         }
204     }
205     else
206     {
207         sub_65B454(v85, a00000000_2);

```

Рисунок 5 – Аналогичный фрагмент для символа секрета

b) Побитовая операция XOR.

Полученные числовые значения преобразуются в двоичное представление, после чего над ними выполняется побитовая операция XOR. Декомпилятор представляет эту

операцию в виде сложной логической конструкции, которая фактически реализует XOR для каждого бита.

```
226     }
227     sub_65B454(v97, &unk_65D302);
228     v59 = v58 + 1;
229     for ( n = 1; n < 9; ++n )
230     {
231         v8 = (__int16 *)sub_404308((System::UnicodeString *)v100, n);
232         v9 = (_DWORD *)sub_404370(v80, *v8);
233         v45 = sub_4244FC(*v9); // Получаем n-ый бит из двоичной строки пароля
234         System::UnicodeString::~UnicodeString((System::UnicodeString *)v80);
235         v10 = (__int16 *)sub_404308((System::UnicodeString *)v98, n);
236         v11 = (_DWORD *)sub_404370(v79, *v10);
237         v44 = sub_4244FC(*v11); // Получаем n-ый бит из двоичной строки секрета
238         System::UnicodeString::~UnicodeString((System::UnicodeString *)v79);
239         if ( (v45 || v44) && (v45 != 1 || v44 != 1) )// Эта сложная логическая конструкция является эквивалентом операции XOR
240         {
241             sub_404330(v76, 1);
242             v13 = unknown_libname_487(v75);
243             System::UnicodeString::operator+(v13, v97, v76);
244             sub_65B524(v97, v75);
245             System::UnicodeString::~UnicodeString((System::UnicodeString *)v75);
246             System::UnicodeString::~UnicodeString((System::UnicodeString *)v76);
247         } // Результат бита равен 1
248     }
249     {
250         sub_404330(v78, 0);
251         v12 = unknown_libname_487(v77);
252         System::UnicodeString::operator+(v12, v97, v78);
253         sub_65B524(v97, v77);
254         System::UnicodeString::~UnicodeString((System::UnicodeString *)v77);
255         System::UnicodeString::~UnicodeString((System::UnicodeString *)v78);
256     } // Результат бита равен 0
257 }
258 v14 = (__int16 *)sub_404308((System::UnicodeString *)v97, 1);
259 v15 = (_DWORD *)sub_404370(v74, *v14);
260 v16 = sub_4244FC(*v15) << 7;
```

Рисунок 6 – Реализация операции XOR

с) Сборка результата.

Результат XOR (новая двоичная строка) преобразуется обратно в десятичное число, которое является ASCII-кодом символа зашифрованной строки. Этот символ добавляется к итоговому буферу unk_65D220.

3) Финальное сравнение:

После завершения цикла, полученная 8-символьная зашифрованная строка (unk_65D220) сравнивается с эталоном, хранящимся в глобальной переменной unk_669638.


```

295     System::UnicodeString::~UnicodeString((System::UnicodeString *)v66);
296     v58 = v59 - 1;
297     System::UnicodeString::~UnicodeString((System::UnicodeString *)v97);
298 }
299 if ( (unsigned __int8)sub_65B5C0(&unk_65D220, &unk_669638) )// sub_65B5C0 - это аналог функции strcmp (сравнение строк)
300 {
301     // Пароль верный, показать вторую форму
302     unknown_libname_487(v64);
303     Vcl::Controls::TControl::GetText(*(Vcl::Controls::TControl **)(a1 + 964));
304     sub_52E684(*(Vcl::Controls::TControl **)(*_DWORD *)off_668E50 + 964));
305     System::UnicodeString::~UnicodeString((System::UnicodeString *)v64);
306     Vcl::Forms::TCustomForm::Show(*(Vcl::Forms::TCustomForm **off_668E50);
307 }
308 else
309 {
310     // Пароль неверный, вывести сообщение
311     v38 = (_DWORD *)sub_65B454(v63, &unk_65D303);
312     unknown_libname_5177(*v38);
313     System::UnicodeString::~UnicodeString((System::UnicodeString *)v63);
314     if ( ++dword_65D228 > 2 )
315     {
316         dword_65D228 = 0;
317         v41 = (System::Win::Registry::TRegistry *)sub_4FF67C((System::Win::Registry::TRegistry *)dword_4FE588);
318         unknown_libname_4004(v41);
319         sub_65B454(v62, aMicroSoftware);
320         unknown_libname_4007(v41);
321         System::UnicodeString::~UnicodeString((System::UnicodeString *)v62);
322         sub_65B454(v60, asc_65D328);
323         v39 = (_DWORD *)sub_65B454(v61, aSoft);
324         System::Win::Registry::TRegistry::WriteString(v41, *v39);
325         System::UnicodeString::~UnicodeString((System::UnicodeString *)v60);
326         System::UnicodeString::~UnicodeString((System::UnicodeString *)v61);
327         Vcl::Forms::TCustomForm::Show(*(Vcl::Forms::TCustomForm **off_668E54);
328         sub_62AA38(Form1);
329         (*(void (__fastcall **)(Vcl::Controls::TControl *, _DWORD))(*(_DWORD *)Form1 + 136))(Form1, 0);
330     }
331 }
332 System::UnicodeString::~UnicodeString((System::UnicodeString *)v98);

```

Рисунок 7 – Финальное сравнение зашифрованной строки с эталоном

Анализ перекрестных ссылок на переменную unk_669638 показал, что она инициализируется строкой INaQI=9D в функции sub_40304C, что подтверждает нашу первоначальную гипотезу.

1.3 Восстановление пароля

На основе проведенного анализа алгоритма шифрования была выведена математическая формула для восстановления исходного пароля.

Формула шифрования, применяемая программой:

$$\text{Код_Символа_Эталона} = (\text{Код_Символа_Пароля} - 48) \wedge (\text{Код_Символа_Секрета} - 48)$$

Поскольку побитовая операция XOR (^) обратима сама к себе (если $A \wedge B = C$, то $A = C \wedge B$), можно выразить из этой формулы код символа исходного пароля:

Формула для расшифровки:

$$\text{Код_Символа_Пароля} = (\text{Код_Символа_Эталона} \wedge (\text{Код_Символа_Секрета} - 48)) + 48$$

Для автоматизации вычислений по данной формуле был разработан Python-скрипт, использующий известные нам данные. (Приложение А)

Процесс восстановления пароля:							
%	Эталон	Код	Секрет	Код-48	XOR	Код+48	Пароль
1	'I'	73	'p'	64	9	57	'9'
2	'H'	72	'x'	72	0	48	'0'
3	'A'	65	'4'	4	69	117	'u'
4	'q'	113	'w'	71	54	102	'f'
5	'I'	73	'p'	64	9	57	'9'
6	'='	61	'4'	4	57	105	'i'
7	'9'	57	'6'	6	63	111	'o'
8	'D'	68	'6'	6	66	114	'r'

Восстановленный пароль: 90uf9ior

Рисунок 8 – Результат работы скрипта: пошаговое вычисление и итоговый пароль

Таким образом, искомый пароль был успешно восстановлен и равен 90uf9ior.

1.4 Проверка результата

Программа успешно приняла пароль, что подтверждает правильность восстановленного алгоритма.

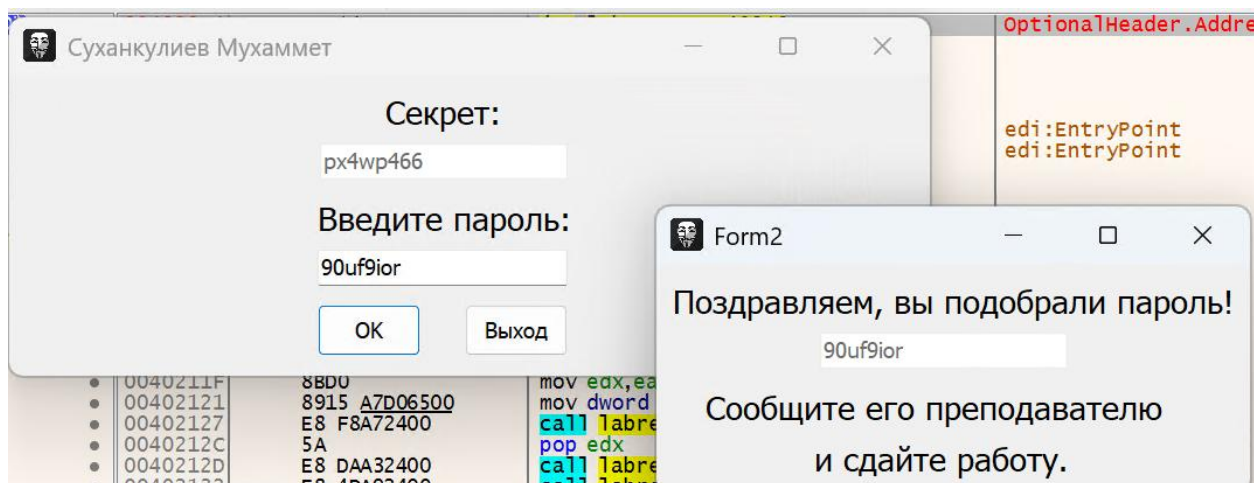


Рисунок 9 – Успешная аутентификация в программе с использованием восстановленного пароля

1.5 Выполнение дополнительных заданий

1.5.1 Вход с любым паролем (x32dbg)

Целью данного этапа было сделать так, чтобы приложение принимало любой пароль.

Временный патч условного перехода `je labreverse.4040C6` по адресу `0x00404052` заменён на `nop`. Результат: любая 8-символьная строка проходит проверку.

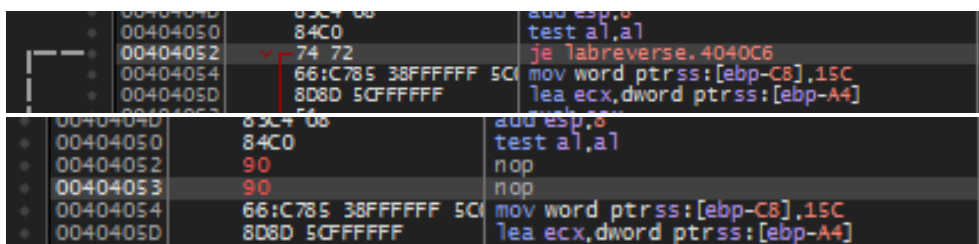


Рисунок 10 – Попытка обхода функции сравнения (до/после)

При вводе строки длиной $\neq 8$ возникала ошибка `Range Check Error`. Причина: цикл шифрования жёстко рассчитан на 8 итераций и обращается к несуществующим символам.

Для обхода этого ограничения была предпринята попытка полного исключения из потока выполнения как проверки длины, так и цикла шифрования. С этой целью в самое начало функции проверки пароля, по адресу `0x004033A7`, была добавлена инструкция безусловного перехода `jmp 404054`, направляющая исполнение сразу на ветку успешной аутентификации.

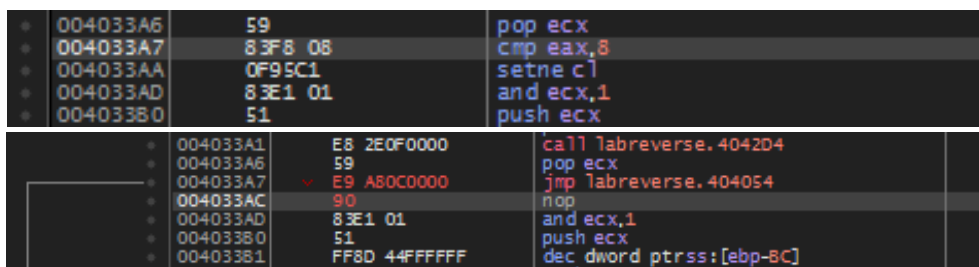


Рисунок 11 – Попытка обхода всех проверок с помощью безусловного перехода (до/после)

Результат: окно успеха появляется, но также выводится окно `Access Violation`. Причина: пропущена инициализация данных/объектов, необходимая для корректной работы ветки успеха.

Вывод. Простая замена ветвлений позволяет пройти проверку, но безопасный путь требует инициализации объектов, поэтому полноценный обход (без ошибок) требует аккуратного патчинга и/или корректной инициализации пропущенных блоков.

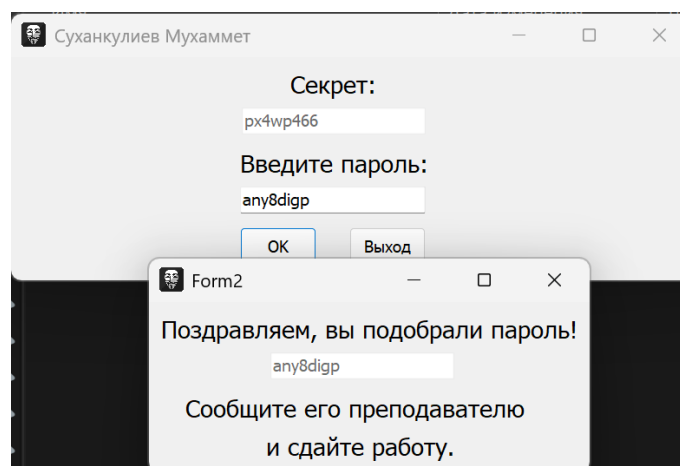


Рисунок 12 – Проверка любого 8 символьный пароля

1.5.2 Снятие блокировки после неверных попыток ввода

Цель: Отключить механизм блокировки после N неверных попыток.

Найден инкремент счётчика неверных попыток: `inc dword ptr ds:[65D228]` по адресу `0x00404107`. Адрес состояния в памяти: `0x0065D228`.

Блокировка выполняется при `cmp dword ptr ds:[65D228], 2 / jle labreverse.404237` (см. `0x0040410D` и `0x00404114`). Если счётчик `> 2` – выполняется код блокировки.

Листинг 1 – Блок кода для проверки количества попыток

```
0040410D | cmp dword ptr ds:[65D228], 2 ; Сравнить счетчик с числом 2
00404114 | jle labreverse.404237 ; Если меньше или равно, перейти к
концу функции
; ... здесь начинается код, отвечающий за блокировку ...
```

Модификация.

Инструкция `jle` по `0x00404114` заменена на безусловный переход `jmp` на тот же адрес назначения. Результат: ветка блокировки пропускается всегда.

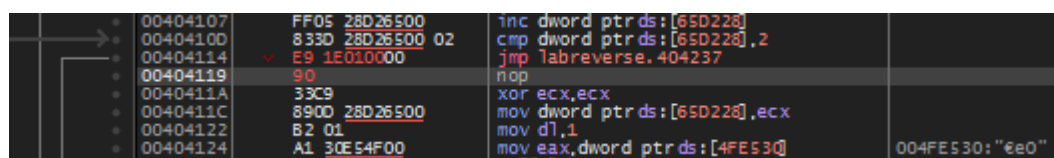


Рисунок 13 – Замена условного перехода на безусловный для обхода блокировки

После патча ограничение снято. Можно выполнять неограниченное количество попыток без срабатывания блокировки.

1.5.3 Разработка алгоритма подбора пароля (Брутфорс)

После снятия блокировки была реализована автоматизация подбора пароля методом «слепого» брутфорса (black-box).

Использована библиотека `pywin32` для управления окном приложения и его контролами (`WM_SETTEXT`, `BM_CLICK`). Это позволяет взаимодействовать напрямую через WinAPI, без имитации пользовательских кликов.

Скрипт находит хендлы главного окна и полей ввода по классам `TForm1`, `TEdit`, `TButton`.

Генерирует кандидаты паролей и подставляет их в поле ввода. Для каждой попытки нажимается кнопка проверки.

После нажатия скрипт проверяет наличие окна успеха (`TForm2`) через `FindWindow` + `IsWindowVisible`. Если успеха нет – закрывает окно ошибки нажатием `Enter` и продолжает.

Листинг 2 – `bruteforce.py`

```
import win32gui
import win32con
import win32api
import time
import itertools

MAIN_WINDOW_CLASS = "TForm1"
INPUT_CLASS_NAME = "TEdit"
BUTTON_CLASS_NAME = "TButton"
SUCCESS_WINDOW_CLASS = "TForm2"

def start_brute_force_test():
    print("Старт...")

    # Поиск хендлов главного окна и его элементов управления
    hwnd_main = win32gui.FindWindow(MAIN_WINDOW_CLASS, None)
    if not hwnd_main:
        print(f"ОШИБКА: {MAIN_WINDOW_CLASS} не найден")
        return

    try:
        h_edit_ro = win32gui.FindWindowEx(hwnd_main, 0, INPUT_CLASS_NAME,
                                           None)
        h_input = win32gui.FindWindowEx(hwnd_main, h_edit_ro,
                                         INPUT_CLASS_NAME, None)
        h_btn_exit = win32gui.FindWindowEx(hwnd_main, 0, BUTTON_CLASS_NAME,
                                           None)
        h_btn_check = win32gui.FindWindowEx(
            hwnd_main, h_btn_exit, BUTTON_CLASS_NAME, None
        )
        if not h_input or not h_btn_check:
            raise ValueError()
    except Exception:
        print("ОШИБКА: Не удалось найти все элементы управления.")
        return
```

```

print("Начинаю перебор...")

# Настройка параметров для ускоренного теста
prefix = "90" # 90uf9ior
charset = "uf9ior"
suffix_length = 6

print(f"Префикс: '{prefix}'")
print(f"Алфавит для перебора: '{charset}'")
start_time = time.time()
attempts = 0
total_combinations = len(charset) ** suffix_length
print(f"Количество комбинаций для проверки: {total_combinations:,}")

# Основной цикл перебора
for p_tuple in itertools.product(charset, repeat=suffix_length):
    suffix = "".join(p_tuple)
    password_candidate = prefix + suffix
    attempts += 1

    # Устанавливаем текст в поле ввода
    win32api.SendMessage(h_input, win32con.WM_SETTEXT, 0,
        password_candidate)

    # Нажимаем кнопку "ОК"
    win32api.PostMessage(h_btn_check, win32con.BM_CLICK, 0, 0)

    time.sleep(0.02)

    # Проверяем, появилось ли видимое окно успеха
    h_success = win32gui.FindWindow(SUCCESS_WINDOW_CLASS, "Form2")
    if h_success and win32gui.IsWindowVisible(h_success):
        end_time = time.time()
        print(f"\nПопытка #{attempts}: '{password_candidate}' -> УСПЕХ!")
        print(f"Затрачено времени: {end_time - start_time:.2f} секунд")
        win32api.SendMessage(h_success, win32con.WM_CLOSE, 0, 0)
        return
    else:
        # Если успеха нет, закрываем окно ошибки нажатием Enter
        win32api.keybd_event(win32con.VK_RETURN, 0, 0, 0)
        win32api.keybd_event(win32con.VK_RETURN, 0,
            win32con.KEYEVENTF_KEYUP, 0)
        time.sleep(0.02)

if __name__ == "__main__":
    try:
        start_brute_force_test()
    except KeyboardInterrupt:
        print("\nПроцесс прерван пользователем.")

```

Для ускоренной демонстрации работоспособности был проведен тестовый запуск, в котором перебирались только последние три символа пароля (`suffix_length = 3`) с использованием сокращенного алфавита. Тест успешно нашел верную комбинацию, что подтверждает корректность алгоритма.

```
C:\Users\muham\Desktop\itmo\_Studying(5th_sem)\ProgrammingTechnologiesAndMethods\лр4>.2.py
Старт...
Начинаю перебор...
Префикс: '90'
Алфавит для перебора: 'uf9ior'
Количество комбинаций для проверки: 46,656
Попытка 500/46,656: '90uu9frf' | Скорость: 24 п/с
Попытка 1,000/46,656: '90uuoioi' | Скорость: 24 п/с
Попытка 1,500/46,656: '90ufurir' | Скорость: 24 п/с

Попытка #1866: '90uf9ior' -> УСПЕХ!
ПАРОЛЬ НАЙДЕН! 90uf9ior
Потребовалось попыток: 1,866
Затрачено времени: 78.14 секунд
```

Рисунок 14 – Скриншот выполнения брутфорса

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был успешно проведен реверс-инжиниринг исполняемого файла `LabReverse.exe`. Путем комбинированного использования статического (IDA Pro) и динамического (x32dbg) анализа была локализована функция проверки пароля и полностью восстановлен алгоритм шифрования, основанный на побитовой операции XOR. На основе полученных данных был вычислен корректный пароль – `90uf9ior`, что позволило получить доступ к программе.

Дополнительные задания позволили углубить понимание механики работы программы и практики её модификации:

Было показано, что за счет изменения отдельных условных переходов можно добиться входа с любым паролем.

Изучен механизм блокировки после неверных попыток ввода. С помощью модификации условного перехода удалось снять это ограничение, что открыло возможность для проведения брутфорса.

Разработан и протестирован алгоритм автоматизированного перебора пароля методом bruteforce через WinAPI.

Работа позволила не только закрепить навыки анализа и восстановления алгоритмов проверки паролей, но и продемонстрировала практические приёмы модификации исполняемого кода, обхода защитных механизмов и разработки вспомогательных инструментов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии программирования: Программа. Процесс обработки данных. Программное средство – Лекция 4: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. Ищенко А. П. Технологии программирования: Модули – Лекция 5: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
3. Eldad Eilam. IDA Pro. The Unofficial Guide to the World's Most Popular Disassembler / Пер. с англ. В. Л. Верещагина. – М.: 2005. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг А.1 – script.py

```
def solve_password():
    # Секрет, который дает программа
    secret = "px4wp466"

    # Эталонная строка, которую мы нашли в отладчике
    correct_encrypted_string = "IHAqI=9D"

    original_password = ""

    print("Процесс восстановления пароля:")
    print("-" * 50)
    print("№ | Эталон | Код | Секрет | Код-48 | XOR | Код+48 | Пароль")
    print("-" * 50)

    for i in range(8):
        # Код i-го символа эталонной зашифрованной строки
        encrypted_char = correct_encrypted_string[i]
        encrypted_char_code = ord(encrypted_char)

        # Код i-го символа секрета, из которого вычли 48
        secret_char = secret[i]
        secret_char_code_part = ord(secret_char) - 48

        # Выполняем обратную операцию XOR
        xor_result = encrypted_char_code ^ secret_char_code_part

        # Прибавляем 48, чтобы получить исходный код символа пароля
        original_char_code = xor_result + 48

        # Преобразуем код обратно в символ
        original_char = chr(original_char_code)

        # Добавляем символ к результату
        original_password += original_char

        print(f"{i+1} | '{encrypted_char}' | {encrypted_char_code:<3} | '{secret_char}' | {secret_char_code_part:<6} | {xor_result:<3} | {original_char_code:<6} | '{original_char}'")

    print("-" * 50)
    print(f"\nВосстановленный пароль: {original_password}")

solve_password()
input()
```